Reti di calcolatori

Alessandro Carella

Indice

1. Reti di calcolatori e Internet 3
   1.1. Storia di Internet 3
   1.2. Gli «ingranaggi» di Internet 3
      1.2.1. I protocolli 4
   1.3. I componenti della rete 4
      1.3.1. Le reti di acceso 4
   1.4. Il nucleo della rete 4
      1.4.1. Commutazione di pacchetto 4
      1.4.2. Commutazione di circuito 5
      1.4.3. Una rete di reti 7
   1.5. Tipi di reti 7
   1.6. Ritardi, perdite e throughput nelle reti a commutazione di pacchetto ... 8
      1.6.1. Ritardo nelle reti a commutazione di pacchetto 8
2. Architetture a livelli 9
   2.1. Stratificazione dei protocolli 9
   2.2. Incapsulamento 12
3. Livello di applicazione 12
   3.1. Web e HTTP 12
      3.1.1. Architettura del Browser e del Server 13
      3.1.2. Differenza tra URI, URN e URL 13
      3.1.3. Panoramica di HTTP 13
      3.1.4. HTTP/1 14
      3.1.5. Connessioni persistenti e non persistenti 15
      3.1.6. Formato dei messaggi HTTP 16
      3.1.7. Interazione utente-server: i cookie 19
      3.1.8. Web caching 20
      3.1.9. HTTP/2 21
      3.1.10. HTTP/3 21
      3.1.11. Streaming video e reti per la distribuzione di contenuti 22
   3.2. DNS: il servizio directory di Internet 26
      3.2.1. Servizi forniti da DNS 26
      3.2.2. Panoramica del funzionamento di DNS 28
      3.2.3. Record e messaggi DNS 31
      3.2.4. Registrazione di un dominio e Zone Transfer 33

Reti di calcolatori e Internet

1.1. Storia di Internet

1.2. Gli «ingranaggi» di Internet

Internet è una rete di calcolatori che interconnette miliardi di dispositivi di calcolo in tutto il mondo. Spesso ci si connette a Internet tramite smartphone e tablet, anche se, oggi, molti più dispositivi come TV, console di gicoo, automobili, elettrodomestici, ecc. vengono via via connesse. Tutti questi dispositivi sono detti **host** o **sistemi periferici** (*end systems*). I sistemi periferici sono connessi tra loro tramite una **rete di collegamenti** (*communication link*) e **commutatori di pacchetti** (*packet switch*). I collegamenti possono essere di molti tipi, costituiti da varie tipologie di mezzi fisici, come cavi coassiali. fili di rame, fibre ottiche, ecc. Collegamenti diversi trasmettono dati a velocità differenenti, e tale **velocità di trasmissione** viene misurata in but/secondo (bps). Quando un host vuole inviare dati a un altro sistema periferico, suddivide i dati in sottoparti e aggiunge un'intestazione a ciascune di esse: l'insieme delle informazioni risultati, viene chiamato **pacchetto**. Un commutatore di pacchetto prende un pacchetto che arriva da uno dei collegamenti in ingresso e lo ritrasmette su uno di quelli in uscita. I princiapli commutatori di pacchetto sono i **router** e i **commutatori a livello di collegamento** (*link-layer switch*). Dal sistema di invio a quello di ricezione, la sequenza di collegamenti e di commuttatori di pacchetto attraversata dal singolo pacchetto è nota come **percorso** (*route* o *path*) attraverso la rete.
I sistemi perferici accedono a Internet tramite i cosiddetti **Internet Service Provider (ISP)**. Un provider è un insieme di commutatori di pacchetto e di collegamenti. GLi ISP forniscono agli host svariati tipi di acceso alla rete, tra cui quello residenziale a larga banda come la DSL, quello in rete locale ad alta velocità e quello wireless.
Sistemi perferici, commutatori di pacchetto e altre parti di Internet fanno uso di protocolli che controllano l'invio e la ricezione di informazioni all'interno della rete. Due dei principali protocolli Internet sono il **Transmission Control Protocol (TCP)** e l'**Internet Protocol (IP)**. Quyest'ultimo specifica il formato dei pacchetti scambiati tra router e sistemi periferici. I principali protocolli Internet sono noti con il nome collettivo di **TCP/IP**.

1.2.1. I protocolli

Diamo una definizione specifica di un protocollo di rete.

Definizione 1. (Protocollo di rete) Un **protocollo** di rete definisce il formato e l'ordine dei messaggi scambiati tra due o più entità in comunicazione, così come le azioni intrapese in fase di trasmissione e/o di ricezione di un messaggio o di un altro evento.

Internet, e le reti di calcolatori in generale, fanno un uso estensivo di protocolli. Si impiegano protocolli differenti per realizzare compiti diversi.

1.3. I componenti della rete

Precedentemente abbiamo visto che nelle reti di calcolatori, i calcolatori e gli atri dispositivi connessi a Internet sono detti **sistemi periferici** o **end system**, in quanto si trovano ai confini di Internet.
I sistemi periferici vengono anche detti **host** quando ospitano programmi applicativi quali browser, web browser o software di lettura e getsione della posta elettronica. Gli host, inoltre, vengono suddivisi in due categorie: **client** e **server**. I client sono host che richiedono servizi e tendono ad essere PC, laptop, smartphon e cosi via, mentre i server si occupano di erogare dei servizi e sono sostanzialmente macchine più potenti che memorizzano e distribuiscono pagine web o flussi video, ritrasmettono la posta elettronica e così via.

1.3.1. Le reti di acceso

Esaminiamo ora le **reti di access**, detti *access network*, cioè la rete che connette fisicamente un sistema al suo **edge router**, che è il primo router sul percorso dal sistema d'origine a un qualsiasi altro sistema di desetinazione collocato al di fuori della stessa rete di accesso.

1.4. Il nucleo della rete

Esaminati i confini di Internet, approfondiamo il nucleo della rete: una maglia di commutatori di pacchetti e collegamenti che interconnettono i sistemi periferici di Internet.

1.4.1. Commutazione di pacchetto

Le applicazioni distribuite scambiano **messaggi** che possono contenere qualsiasi cosa il progettista del protocollo desideri. Possono svolgere una funzione di controllo o contenere dati come in un messaggio di posta elettronica, un'immagine JPEG o un file audio. La sorgente suddivide i messaggi lunghi in parti più piccole note come **pacchetti**. Tra la sorgente e la destinazione, questi pacchetti viaggiano attraverso

collegamenti e **commutatori di pacchetto**. I pacchetti vengono trasmessi su ciascun collegamento a una velocità pari alla velocità totale di trasmissione del collegamento stesso.

1.4.2. Commutazione di circuito

Per spostare i dati in una rete di collegamenti e commutatori esistono due approcci fondamentali: la **commutazione di circuito** e la **commutazione di pacchetto**. Nelle reti a commutazione di circuito le risorse richieste lungo un perocrso per consentire la comunicazione tra sistemi periferici sono riservate per l'intera durata della sessione di comunicazione. Nelle reti a commutazioni di pacchetto, tali risorse non sono riservate; i messaggi di una sessione utilizzano le risorse quando necessario, e di conseguenza potrebbero dover attendere per accedere a un collegamento.
Le reti telefoniche sono esempi di reti a commutazione di circuito. Si consideri che cosa avviene quando una persona vuole inviare informaizoni a un altro soggetto. Prima che possa iniziare l'invio, la rete deve stabilire una connessione tra mittente e destinatario. Questo è un collegamento «a regola d'arte» in cui i commutatori sul percorso tra mittente e destinatario mantegono lo stato della connessione per tutta la durata della comunicazione. Nel gergo della telefonia questa connessione è detta **circuito**.

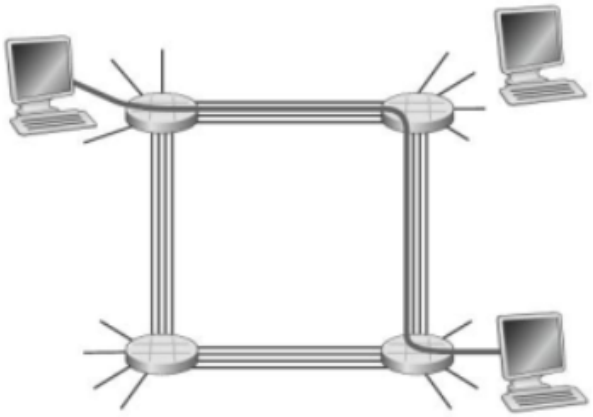
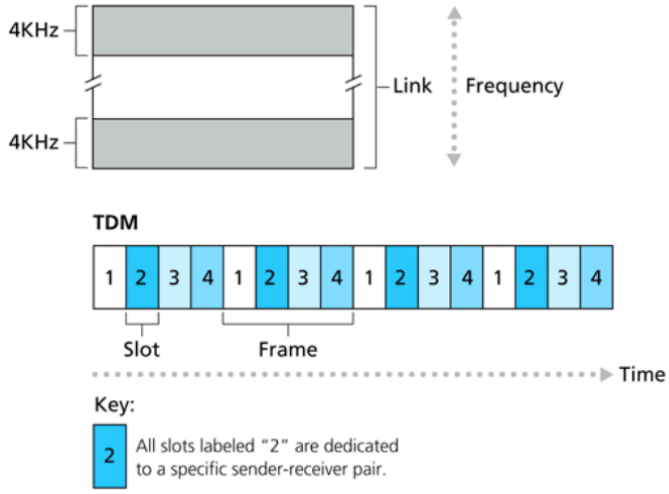


Figura 1: Rete a commutazione di circuito

La Figura 1 mostra una rete a commutazione di circuito in cui i quattro commutatori sono interconnessi tramite quattro colegamenti. Gli host sono tutti direttamente connessi a uno dei commutatori. Quando due host desiderano comunicare le rete stabilisce una **connessione end-to-end** dedicata a loro. Affinchè A invii messaggi a B, la rete deve prima riservare un circuito su ciascuno dei due collegamenti.
Si consideri invece che cosa accade quando un host invia un pacchetto a un altro host su una rete a commutazione di pacchetto, quale è Internet. Come nella commutazione a circuito, il pacchetto viene trasmesso su di una sequenza di collegamenti

ma, a differenza della commutazione di circuito, il pacchetto viene immesso nella rete senza che vengano riservate risorse. Se un collegamento è congestionato perchè vi sono altri pacchetti che devono essere trasmessi nello stesso istante, allora dovrà aspettare nel buffer del lato mittente e subire un ritardo.

Multiplexing nelle reti a commutazione di circuito
Un circuito all'interno di un collegamento è implementato tramite **multiplexing a divisione di frequenza (FDM, *frequency-division multiplexing*)** o **multiplexing a divisione di tempo (TDM, *time_division multiplexing*)**. Con FDM, lo spettro di frequenza di un collegamento viene suddiviso tra le connessioni stabilite tramite il collegamento. Nello specifico, il collegamento dedica una banda di frequenza a ciascun connessione per la durata della connessione stessa. La larghezza della banda viene detta **ampiezza di banda** o *bandwidth*.
Per un collegamento TDM il tempo viene suddiviso in frame di durata fissa, a lroo volta ripartiti in un numero fisso di slot temporali. Quando la rete stabilisce una connessione attraverso un collegamento, le dedica uno slot di tempo in ogni frame. Tali slot sono dedicati unicamente a quella connessione, con uno slot temporale disponibile in ciascun frame alla trasmissione dei dati di connessione.



La Figura 2 mnostra FDM e TDM per uno specifico collegamento di rete che supporta fino a 4 circuiti. Nel caso di FDM il dominio delle frequenze viene ripartito in quattro bande, ciascuna con ampiezza di 4 kHz. Nel caso di TDM, il dominio del tempo viene suddiviso in frame, con quattro slot di tempo per ciascun intervallo; a ogni circuito viene assegnato lo stesso slot dedicato in tutti i frame. Nel caso di TDM, la velocità di trasmissione di un circuito è uguale alla frequenza di frame moltiplicata per il numero di bit in uno slot.
Un'ulteriore strategia di multiplazione è la **WDM, Wavelength-Division Mul-**

tiplexing che viene usata per connessioni e fibra ottica e prevede una multiplazione e demultiplazione dei flussi che sfruttano diverse lunghezze d'onda della luce laser.

1.4.3. Una rete di reti

Abbiamo visto che i sistemi periferici si collegano a Internet tramite un ISP di accesso che può fornire connettività attraverso una rete cablata o senza fili. Tuttavia, la connessione degli utenti finali e dei fornitori di contenuti alla rete di un ISP è solouna piccola parte del puzzle per connettere i miliardi di utenti che costituiscono Internet. Per completare il puzzle bisogna interconnettere gli stessi ISP. Ciò avviene creando una **rete di reti**.
Per connettere gli ISP, un approccio ingenuo sarebbe quello di connettere direttamente ogni ISP di accesso con tutti gli altri, ma una struttura simile, detta a *maglia*, è troppo costosa.
Si adotta, quindi, una struttura gerarchica basata su livelli. Ci sono gli ISP di accesso di livello 3, gli ISP regionali di livello 2 e gli ISP globali di livello 1. Ogni ISP è cliente di uno del livello superiore e il cliente paga il fornitore per il transito verso il resto di Internet.
Un altra soluzione è quella di avere al centro poche decine di ISP di livello 1 e ciascuno è connesso a un gran numero di ISP di livello inferiore.
Il problema di questo tipio di connessione è il costo. Infatti gli ISP di livello inferiore pagano gli ISP di livello superiore e il costo riflette la quantità di traffico che l'ISP cliente scambia con il fornitore. Per ridurre tali costi, una coppia di ISP vicini e di pari livello gerarchico può fare uso di **peering**, cioè connettere direttametne le redi in modo che tutto il traffico tra di esse passi attraverso una connessione diretta. In questo modo due ISP si scambiano traffico senza pagarsi. L'**IXP**, Internet eXchange Point, è la sede fisica dove molte resi si interconnettono e fanno peering.

1.5. Tipi di reti

Esiste una grande varietà di tecnologie di rete e di modelli organizzativi. In particolare, le reti possono essere classificate in base all'estensione geografica:

- Rete personale, dette **PAN (Personal Area Network)**, sono reti in ambitoo personale e connettono apparati personali nel raggio di pochi metri: smartphone, cuffie, smartwatch, stampanti e così via. I collegamenti possono essere guidati tramite USB oppure via wireless;
- Rete locale, dette **LAN (Local Area Network)** e sono reti che si estendono all'interno di un edificio o di un comprensorio, con un'estensione nell'ordine del centinaio di metri. Le LAN hanno velocità trasmissive elevate (1-10 Gbps su cavo). Le LAN oggi sono reti commutate con collegamenti punto-punto e i collegamenti sono guidati e wireless;

- Rete metropolitana, dette **MAN (Metropolitan Area Network)** e sono reti in ambito cittadino. Il mezzo trasmissivo tipico è la fibra ottica sul backcon, oppure cavi in fibra/rame;
- Rete geografica, dette **WAN (Wide Area Network)** e sono reti in ambito nazionale o internazionale. I collegamenti sono indiretti guidati e wireless. Possono essere reti a commutazione di circuito e di pacchetto e le topologie possono essere svariate.

1.6. Ritardi, perdite e throughput nelle reti a commutazione di pacchetto

1.6.1. Ritardo nelle reti a commutazione di pacchetto

Un pacchetto parte da un host, passa attraverso una serie di router e conclude il viaggio in un altro host. A ogni tappa, il pacchetto subisce vari tipi di ritardo a ciascun nodo del tragitto. Di tali ritardi, i principali sono il **ritardo di elaborazione**, il **ritardo di accodamento**, il **ritardo di trasmissione** e il **ritardo di propagazione** che complessivamente formano il **ritardo totale di nodo**. Esaminiamoli nel dettaglio.

- il *ritardo di elaborazione* è il tempo richiesto per esaminare l'intestazione del pacchetto e per determinare dove dirigerlo. Questo può anche includere altri fattori, tra i quali il tempo richiesto per controllare errori a livello di bit.
- il *ritardo di accodamento* è dato dalla somma dei tempo di attesa per ogni coda
- il *ritardo di trasmissione* è dato dal tempo di immissione del pacchetto sul collegamento. È pari al rapporto delle dimensione del pacchetto (in bit) e la velocità di trasmissione (in bps).
- il *ritardo di propagazione* è dato dalla distanza/velocità del segnale sul mezzo trasmissivo.

Inoltre, ci sono altri tipi di ritardo, quali:

- **ritardo end-to-end** che è dato dalla somam dei ritardi impeigato da un pacchetto per andare da un nodo di partenza a quello di destinazione;
- **round trip time** (tempo di andata e ritorno) è il tempo impiegato da un pacchetto per andare da un punto all'altro della rete e tornare al unto di partenza ed è dato da $2 \cdot$ ritardo end-to-end

Per ottenere una misura efficiente dei ritardi utilizziamo due strumenti:

1. *ping* calcola il ritardo rispetto ad una destinazione finale per un messaggio di dimensioni standard;
2. *traceroute (tracert)* calcola il ritardo per ogni nodo intermedio fino alla destinazione finale per un messaggio di dimensioni standard.

Architetture a livelli

Precedentemente abbiamo descritto come è costituita la rete Internet, che è, quindi, un sistema complicato ed esiste un'organizzazione per organizzare l'architettura delle reti.
Un'architettura a livelli consente di discutere una parte specifica e ben definita di un sistema articolato e complesso. Questa semplificazione ha un valore importante grazie all'introduzione della modularità, che rende molto più facile cambiare l'implementazione del servizio fornito da un determinato livello. Fino a quando il livello fornisce lo stesso servizio allo strato superiore e utilizza gli stessi servizi dello strato inferiore, la parte rimanente del sistema rimane invariata al variare dell'implementazione del livello. In sistemi grandi e complessi, che vengono costantemente aggiornati, la capacità di cambiare l'implementazione di un servizio senza coinvolgere altre componenti del sistema costituisce un ulteriore importante vantaggio legato alla stratificazione.

2.1. Stratificazione dei protocolli

Per dare una struttura ai protocolli di rete, i progettisti organizzano protocolli e l'hardware che li implementano in **strati**, detti *layer*. Ciascun protocollo appartiene a uno dei livelli. Adesso rivolgiamo l'attenzione ai **servizi** offerti da un livello superiore: si tratta del cosidetto **modello di servizio**, detto *service model*, di un livello. Un livello di protocolli può essere implementato via software, hardware o con una combinazione dei due. I protocolli a livello di applicazione, quali HTTP e SMTP, sono quasi sempre implementati via software nei sistemi periferici; e così è anche per i protocolli a livello di trasporto. Dato che i livelli fisico e di data link si occupano della comunicazione su un collegamento specifico, sono di regola implementati nella scheda di rete associata. Il livello di rete è spesso un'implementazione mista di hardware e software.
La stratificazione dei protocolli presenta vantaggi concettuali e strutturali. Fornisce un modo strutturato per trattare i componenti dei sistemi. La modularità rende più facile aggiornare la componentistica. Uno svantaggio legato alla stratificazione è la possibilità che un livello duplichi le funzionalità di quello inferiore. Ad esempio, molte pile di protocolli forniscono meccanismi di correzione degli errori ai livelli sia di collegamento sia di connessione end-to-end. Un secondo potenziale svantaggio è che la funzionalità a un livello possa richiedere informazioni presenti solo in un altro livello e questo viola lo scopo insito nella separazione dei livelli.
Considerati assieme, i protocolli dei vari livelli sono detti **pila di protocolli**, detto

protocol stack. La pila di protocolli di Internet, mostrata nella seguente figura, consiste di cinque livelli: fisico, collegamento, rete, trasporto e applicazione.



Figura 3: Pila di protocolli di Internet

Analizziamo brevemente i vari livelli:

- il livello di *applicazione* è la sede delle applicazioni di rete e dei relativi protocolli. Per quanto riguarda Internet, questo livello include molti protocolli, come HTTP che consente la richiesta e il trasferimento dei documenti web, SMTP che consente il trasferimento dei messaggi di posta elettronica e FTP che consente il trasferimento di file tra due sistemi remoti. I pacchetti di informazione vengono chiamati **messaggi**.
- il livello di *trasporto* di Internet trasferisce i messaggi del livello di applicazione tra punti periferici gestiti dalle applicazioni. In Internet troviamo due protocolli di trasporto: TCP e UDP. TCP fornisce alle applicazioni un servizio orientato alla connessione, che include la consegna garantita dei messaggi a livello di applicazione alla destinazione e il controllo di flusso. Inoltre, TCP fraziona i messaggi lunghi in segmenti più piccoli e fornisce un meccanismo di controllo della congestione.
  Il protocollo UDP fornisce alle proprie applicazioni un servizio non orientato alla connessione che è un servizio senza affidabilità, né controllo di flusso e della congestione. I pacchetti a livello di trasporto si chiamano **segmenti**.
- Il lviello di rete di Internet si occupa di trasferire i pacchetti a livello di rete, detti **datagrammi**, da un host a un altro. Il livello di rete di Internet comprende il famoso protocollo IP, che definisce i campi dei datagrammi e come si sistemi periferici e i router agiscono su tali campi. Il livello di rete di Internet contiene svariati protocolli di instradamento che determinano i percorsi che i datagrammi devono seguire tra la sorgente e la destinazione.
- Il livello di *collegamento* si occupa di trasferire il datagramma al livello di rete superiore. I servizi forniti da questo livello dipendono dallo specifico protocollo utilizzato. Esempi di livello di collegamento includono Ethernet, Wi-fi e il proto-

collo di accesso alla rete DOCSIS. I pacchetti a livello di collegamento vengono chiamati **frame**.

- Il livello *fisico* è trasferire i singoli bit del frame da un nodo a quello successivo. Anche i protocolli di questo livello dipendono dal collegamento e in più dipendono dall'effettivo mezzo trasmissivo.

Un architettura a livelli alternativa al modello presentato prima, detto modello TCP/IP è il modello ISO/OSI, presentato nella seguente figura:



Figura 4: Modello ISO/OSI

ISO sta per International Organization for Standardization e OSI per Open Systems Interconnection. Il modello ISO/OSI è un sistema aperto con caratteristiche di interoperabilità. Esso è composto da 7 livelli:

1. il livello *fisico* che si occupa di trasmetter bit lungo un canale di comunicazione;
2. il livello di *collegamento* che si occupa di trasmettere i frame tra nodi adiacenti collegati direttamente, con rilevazione degli errori;

3. il livello di *rete* che si occupa di instradare i datagrammi tra reti a commutazione di pacchetto;
4. il livello di *trasporto* che si occupa di far comunicare, in modalità end-to-end, i processi trasmettendo segmenti;
5. il livello di *sessione* che si occupa di organizzare il dialogo di sincronizzare i programmi applicativi;
6. il livello di *presentazione* che si occupa di come rappresentare i dati da trasferire;
7. il livello di *applicazioni* che si occupa del trasferimento di messaggi tra le parti di una applicazione.

2.2. Incapsulamento

Quando un messaggio attraversa i vari livelli della rete si applica il cosiddetto meccanismo di **incapsulamento**. Presso un host mittente, un **messaggio a livello di applicazione**, *application-layer message*, viene passato a livello di trasporto. Questo livello prende il messaggio e gli concatena informazioni aggiuntine, dette informazioni di intestazione che saranno utilizzate dalla parte ricevente del livello di trasporto. Il messaggio a livello di applicazione e le informazioni di intestazione a livello di trasporto costituiscono il **segmento a livello di trasporto**, *transport-layer segment* che incapsula il messaggio a livello di applicazione. Le informazioni aggiunte potrebbero includere dati che consentono al livello di trasporto di consegnare il messaggio all'applicazione desiderata. Il livello di trasporto passa il segmento al livello di rete, che aggiunge informazioni di intestazioni proprie, quali gli indirizzi dei sistemi perfiferici di sorgente e di destinazione, creando un **datagramma a livello di rete**, *network-layer datagram*. A questo punto, il datagramma viene passato al livello di collegamento, il quale aggiunge le proprie informazioni di intestazioni creando un **frame a livello di collegamento**, *link-layer frame*. Quindi a ciascun livello, il pacchetto ha due tipi di campo: quello di intestazione e quello di **payload** (il carico utile trasportato). Il payload è tipicamente un pacchetto proveniente dal livello superiore.

Livello di applicazione

3.1. Web e HTTP

Fino agli anni "90 Internet veniva principalmente utilizzata da ricercatori, docenti e studenti universati per raggiungere host remoti, trasferire file, ricevere e inviare

notizie e per la posta elettronica. Poi, comparve il World Wide Web. Il Web è stata la prima applicazione Internet che ha cambiato profondamente il modo di interagire all"interno e all'esterno degli ambienti di lavoro.
Ciò che attira del Web è che opera su richiesta , si può avere ciò che si vuole, quando lo si vuole. Oltre a questo, il Web presenta molte altre interessanti caratteristiche. È facilissimo per tutti rendere disponibili informazioni sul Web: chiunque può diventare editore a costi estremamente bassi. Inoltre il Web fornisce una piattaforma a molte applicazioni killer, tra cui YouTube, servizi di posta elettronica come Gmail, Instagram e Google Maps.

3.1.1. Architettura del Browser e del Server

L'architettura di un web browser comprende diversi moduli specializzati: un Display manager, la gestione delle Estensioni/Plug-in, un Cache manager, un Request processor, il motore JavaScript (JS engine), un HTML interpreter e lo stack di rete HTTP/TCP.
Parallelamente, l'architettura di un web server (o **http daemon**) svolge le funzioni di gestire le connessioni con i browser e ritrovare o creare documenti dal File System. Include anche un modulo per l'interpretazione CGI/HTTP, un manager delle richieste e un Path Resolver per gestire le autorizzazioni di accesso. I web server oggi più diffusi sul mercato sono Apache, Nginx, Cloudflare e LiteSpeed, con IIS ormai marginale.

3.1.2. Differenza tra URI, URN e URL

Il sistema di indirizzamento del Web si basa formalmente sugli URI (Uniform Resource Identifier), definiti nella RFC 3986, che identificano una risorsa attraverso il nome o la posizione. Gli URI si dividono principalmente in due categorie:

- **URN (Uniform Resource Name):** Associa un identificatore univoco alla risorsa senza implicarne la disponibilità e senza specificare il meccanismo di accesso (es. un codice ISBN per un libro).
- **URL (Uniform Resource Locator):** Specifica sia il metodo di accesso sia la locazione fisica della risorsa, dandone per implicita la disponibilità; se la locazione cambia, il link si rompe («broken link»).

La sintassi generale di un URL è strutturata in questo modo: `scheme:["//"] [user [":"password] "@"] host [":"port] ["/"url-path]` . Lo **scheme** indica il protocollo (es. http, https, mailto, file), l'**host** è il nome DNS o l'indirizzo IP, la **port** è il numero di porta (opzionale, default 80 per HTTP o 443 per HTTPS) e l'**url-path** è il percorso gerarchico della risorsa seguito da eventuali parametri di query.

3.1.3. Panoramica di HTTP

HTTP, acronimo di *HyperText Transfer Protocol*, protocollo a livello di applicazione del Web, costituisce il cuore del Web. Questo protocollo è implementato in due

programmi, client e server, in esecuzione su sistemi periferici diversi che comunicano tra loro scamiandosi messaggi HTTP. Il protocollo definisce sia la struttura dei messaggi sia la modalità con cui client e server si scambiano i messaggi.
Una **pagina web**, detta anche documento, è costituita da oggetti. Un **oggetto** è semplicemente un file indirizzabile tramite un URL. La maggioranza delle pagine web consiste di un **file HTML principale** e diversi oggetti referenziati da esso. Ogni URL ha due componenti: il nome dell'host del server che ospita l'oggetto e il percorso dell'oggetto. Per esempio, l'URL

http.//www.someSchool.edu/someDepartment/picture.gif

ha *www.someSchool.edu* come nome dell'host e */someDepartment/picture.gif* come percorso. Un **browser web** implementa il lato client di HTTP. Un **web server**, che implementa il lato server di HTTP, ospita oggetti web, indirizzabili tramite URL.
HTTP definisce in che modo i client web richiedono le pagine ai web server e come questi ultimi le trasferiscono ai client. Quando l'utente richiede una pagina web, il browser invia al server messaggi di richiesta HTTP per gli oggetti nella pagina. Il server riceve le richieste e risponde con messaggi di risposta HTTP contenenti gli oggetti.
HTTP utilizza TCP (anzichè UDP) come protocollo di trasporto. Il client HTTP per prima cosa inizia una connessione TCP con il server. Una volta stabilita, i processi client e server accedono a TCP attraverso le proprie socket. L'interfaccia socket è la porta tra un processo e la sua connessione TCP. Il client invia richieste e riceve risposte HTTP tramite la propria interfaccia socket, analogamente il server riceve richieste e invia messaggi di risposta attraverso la propria interfaccia socket. Quando il client ha mandato un messaggio alla sua interfaccia socket, questo non è più in suo possesso, ma si trova «nelle mani» di TCP. TCP mette a disposizione di HTTP un servizio di trasferimento dati affidabile. Questo è uno dei grandi vantaggi di un'architettura organizzata a livelli: HTTP non si deve preocuppare dei dati smarriti o di come TCP recuperi le perdite o riordini i dati all'interno della rete: questi sono compiti di TCP e dei protocolli di livello inferiore.
Il server invia i file richiesti ai clienti senza memorizzare alcuna informazione di stato a proposito dei client. Per cui, in caso di ulteriore richiesta dello stesso oggetto da parte dello stesso client, anche nel giro di pochi secondi, il server procedera nuovamente all'invio. Dato che i server HTTP non mantengono informazioni sui client, HTTP è classificato come **protocollo senza memoria di stato**. Un web server è sempre attivo, ha un indirizzo IP fisso e risponde potenzialmente alle richieste provenienti da milioni di diversi browser.

3.1.4. HTTP/1

La versione originale di HTTP si chiama HTTP / 1.0 e risale agli inizi degli anni "90. A partire dal 2020, la maggior parte delle transazioni HTTP avviene su HTTP/1.1.

Tuttavia, sempre più browser e web server supportano una nuova versione di HTTP chiamata HTTP/2.

3.1.5. Connessioni persistenti e non persistenti

In molte applicazioni per Internet, client e server comunicano per un lungo periodo di tempo, con il client che inoltra una serie di richieste e il server che risponde a ciascuna di esse. Quando tale interazione ha luogo su TCP, gli sviluppatori dell'applicazione devono prendere una decisione importante: devono decidere se ciascuna coppia richiesta/risposta deve essere inviata su TCP *separata* oppure devono essere inviate tutte sulla stessa connessione TCP. Nel primo approccio si dice che l'applicazione usa **connessioni non persistenti**, mentre nel secondo usa **connessioni persistenti**.

HTTP con connessioni non persistenti

Analizziamo il trasferimento di un file nel caso di connessione non persistente. Supponiamo che la pagina consista di un file HTML principale e di 10 immagini JPEG, e che tutti gli undici oggetti risiedano sullo stesso server. Ipotezziamo che l'URL del file HTML principale sia:

http://wwww.someSchool.edu/someDepartment/home.index

Le fasi della connessione sono le seguenti.

1. Il processo client HTTP inizializza una connessione TCP con il server *www.someSchool.edu* sulla porta 80, che è la porta di default per HTTP. Associate alla connessione TCP ci saranno una socket per il client e una per il server.
2. Il client HTTP, tramite la propria socket, invia al server un messaggio di richiesta HTTP che include il percorso */someDeparment/home.index*.
3. Il processo server HTTP riceve il messaggio di richiesta attraverso la propria socket associata alla connessione, recupera l'oggetto */someDeparment/home.index* dalla memoria, lo incapsula in un messaggio di risposta HTTP che viene inviato al client attraverso la socket.
4. Il processo server HTTP comunica a TCP di chiudere la connessione. Questo, però, non termina la connessione finchè non sia certo che il client abbia ricevuto integro il messaggio di risposta.
5. Il client HTTP riceve il messaggio di risposta. La connessione TCP termina. Il messaggio indica che l'oggetto incapsulato è un file HTML. Il client estrae il file dal messaggio di risposta, esamina il file HTML e trova i riferimenti ai 10 oggetti JPEG.
6. Vengono quindi ripetuti i primi 4 passi per ciascuno degli oggetti JPEG referenziati.Questi passi illustrano l'utilizzo di connessioni non persistenti in cui ogni connessione TCP viene chiusa dopo l'invio dell'oggetto da parte del

server: vale a dire che ciascuna trasporta soltanto un messaggio di richiesta e un messaggio di risposta.

Possiamo fare un calcolo approssimativo per stimare l'intervallo di tempo che intercorre tra la richiesta di un file HTML da parte del client e il momento in cui l'intero file viene ricevuto dal client/ A questo scopo, definiamo il **round-trip time (RTT)**, che rappresenta il tempo impiegato da un singolo pacchetto per viaggiare dal client al server e poi tornare al client. RTT include i ritardi di propagazione, di accodamento nei router e nei commutatori intermedi nonchè di elaborazione del pacchetto. Quando l'utente clicca su un collegamento ipertestuale, invece, questo comporta un **handshake a tre vie**: il client invia un piccolo segment TCP al server e quest'ultimo manda una conferma per mezzo di un piccolo segmento TCP. Infine il client dà anch'esso una conferma di ritorno al server. Le prime due parti dell'handshare a tre vie richiedono un RTT. Dopo il loro completamento, il client invia un messaggio di richiesta HTTP combinano con la terza parte dell'handshake, la conferma di avvenuta ricezione, tramite la connessione TCP>Quando il messaggio di richiesta arriva al server, quest'ultimo inoltra il file HTML sulla connessione TCP. La richiesta-risposta HTTP consuma un altro RTT. Pertanto, il tempo di risposta è approssivamente di due RTT più il tempo di trasmissione da parte del server del file HTML.

HTTP con connessioni persistenti

Le connessioni non persistenti presentano alcuni limiti:

1. per ogni oggetto richiesto occorre stabilire e mantenere una nuova connessione. Per ogni connessione si devono allocare buffer e mantenere variabili TCP sia nel client sia nel server.
2. ciascun oggetto subisce un ritardo di consegna di due RTT, uno per stabilire la connessione TCP e uno per richiedere e ricevere un oggetto.

Con HTTP 1.1 nelle connessioni persistenti il server lascia la connessione TCP aperta dopo l'invio di una risposta, per cui le richieste e le risposte successive tra gli stessi client e server possono essere trasmesse sulla stessa connessione. Inoltre, il server può inviare un'intera pagina web o anche più pagine web allo setsso client.
Queste richieste di oggetti possono essere effettuate una di seguito all'altra senza aspettare le risposte delle richieste pendenti (meccanismo di *pipeling*). In generale, il server HTTP chiude la connessione quando essa rimane inattiva per un dato lasso di tempo. La modalità di default di HTTP impiega connessioni persistenti con pipeling.

3.1.6. Formato dei messaggi HTTP

Le specifiche HTTP includono la definizione dei due formati dei messaggi HTTP, di richiesta e di risposta.

Messaggio di richiesta HTTP

Un messaggio tipico di richiesta HTTP si presenta nel seguente modo:

```
GET /somedir/page.html HTTP/1.1
Host: www.someschool.edu
Connection: close
User-agent: Mozilla/5.0
Accept-language: fr
```

Il messaggio è scritto in testo ASCII, in modo che l'utente sia in grado di leggerlo. Inoltre, consiste di cinque righe, ciascuna seguito da un carattere di ritorno a capo e un carattere di nuova linea aggiuntivi. In generale, i messaggi di richiesta possono essere costituito da un numero indefinito di righe, anche una sola. La prima riga è detta **riga di richiesta** (*request line*), quelle successive **righe di intestazione** (*header lines*). La riga di richiesta presenta tre campi: il campo metodo, il campo URL e il campo versione di HTTP. Il campo metodo può assumere diversi valori, tra cui `GET, POST, HEAD, PUT` e `DELETE`. La maggioranza dei messaggi di richiesta HTTP usa il metodo `GET`, adottato quando il browser richiede un oggetto identificato dal campo URL. La versione è auto esplicativa.
Consideriamo le righe di intestazione dell'esempio. La riga *Host: www.someSchool.edu* specifica l'host su cui risiede l'oggetto. Includendo la linea di intestazione *Connection: close*, il browser sta comunicando al server che non si deve occupare di connessioni persistenti, ma vuole che questi chiuda la connessione dopo aver inviato l'oggetto richiesto. La riga di intestazione *User-agent:* specifica il tipo di browser che sta effettuando la richiesta al server. Questa riga è utile in quanto il server può inviare versioni diverse dello stesso oggetto a browser di tipi diversi. Infine, *Accept-language:* indica che l'utente preferisce ricevere una versione in francese dell'oggetto se disponibile; altrimenti, il server dovrebbe inviare la versione di default.
Dopo le linee di intestazione si trova un «corpo». Nel caso di metodo GET è vuoto, ma viene utilizzato nel metodo POST, il quale viene usato per inviare dati al server. Le richieste generate con il form non devono necessariamente usare il metodo POST. Anzi, spesso i form HTML utilizzano il metodo GET E includono i dati immessi nell'URL richiesto. Ad esempio:

www.somesite.com/animalsearch?scimmie&banane

Il metodo HEAD è simile a GET. Quando un server riceve una richiesta con il metodo HEAD, risponde con un messaggio HTTP, ma tralascia gli oggetti richiesti. Gli sviluppatori spesso utilizzano il metodo HEAD per verificare la correttezza del codice prodotto. Il metodo PUT, consente agli utenti di inviare un oggetto a un percorso specifico su uno specifico web server.

Messaggio di risposta HTTP

Analizziamo un tipico messaggio di risposta HTTP che potrebbe rappresentare la risposta al messaggio di richiesta dell'esempio precedente.

```
HTTP/1.1 200 OK
Connection: close
Date: Thu, 18 Aug 2015 15:44:04 GMT
Server: Apache/2.2.3 (CentOS)
Last-Modified: Tue, 18 Aug 2015 15:11:03 GMT
Content-Length: 6821
Content-Type: text/html
(data data data data data ...)
```

Analizzando questo messaggio di risposta ci sono tre sezioni:

1. una **riga di stato** iniziale. Questa presenta tre campi:
   1. la versione del protocollo
   2. un codice di stato
   3. un corrispettivo messaggio di stato;
2. sei **righe di intestazione**. Il server utilizza la riga di intestazione *Connection: close* per comunicare al client che ha intenzione di chiudere la connessione TCP dopo l'invio del messaggio. La riga *Date:* indica l'ora e la data di creazione e invio, da parte del server, della riposta HTTP. La riga *Server:* indica che il messaggio è stato generato da un web server Apache. La riga *Last-Modified*: indica l'istante e la data in cui l'oggetto è stato creato o modificato per l'ultima volta. La riga di intestazione *Content-Length:* contiene il numero di byte dell'oggetto inviato. La riga *Content-Type* indica che l'oggetto nel corpo è testo HTML. ;
3. il **corpo**: questo è il fulcro del messaggio contiene l'oggetto richiesto.

Analizziamo alcuni codici di stato e le relative espressioni.

1. 200 OK: la richiesta ha avuto successo e in risposta si invia l'informazione;
2. 301 Moved Permanently: l'oggetto richiesto è stato trasferito in modo permanente; il nuovo URL è specificato nell'intestazione *Localtion:* del messaggio di risposta.
3. 400 Bad Request: si tratta di un codcie di errore generico che indica che la richiesta non è compresa dal server;
4. 404 Not Found: il documento richiesto non esiste sul server.
5. 505 HTTP Version Not Supported: il server non dispone della versione di protocollo HTTP richiesta.

Il campo `Content-Type` sfrutta lo standard MIME (Multipurpose Internet Mail Extensions) per indicare la natura dei dati trasmessi. Tra i tipi e sottotipi più comuni troviamo testo (`text/plain` , `text/html` , `text/xml`), immagini (`image/gif` , `image/jpeg`), audio (`audio/basic`), video (`video/mpeg`) e file applicativi (`application/pdf`).

Per quanto riguarda i codici di stato, il protocollo HTTP li suddivide in grandi categorie:

- 2xx (Successful): Include 200 OK, 201 Created (utilizzato per POST/PUT) e 204 No Content.
- 3xx (Redirection): Include 301 Moved Permanently, 302 Found (trasferimento temporaneo) e 304 Not Modified (indica che la cache è ancora valida).
- 4xx (Client Error): Include 400 Bad Request, 401 Unauthorized, 403 Forbidden e 404 Not Found.
- 5xx (Server Error): Include 500 Internal Server Error, 501 Not Implemented, 502 Bad Gateway, 503 Service Unavailable e 504 Gateway Timeout.

Inoltre, per velocizzare il caricamento con connessioni persistenti (HTTP/1.1), va notato che i browser moderni adottano un approccio fuori dallo standard originario (che suggeriva max 2 connessioni), aprendo in parallelo fino a 6 connessioni TCP per ciascun hostname.

3.1.7. Interazione utente-server: i cookie

Abbiamo detto che i server HTTP sono privi di stato. Ciò semplifica la progettazione e consente di sviluppare web server ad alte prestazioni, in grado di gestire migliaia di connessioni TCP simultanee. Spesso i web server possono autenticare gli utenti, sia per limitare l'accesso da parte di questi ultimi sia per fornire contenuti in funzione della loro identità. A questo scopo HTTP adotta i **cookie**. I cooie, consentono ai server di tener traccia degli utenti. La tecnologia dei cookie presenta quattro componenti:

1. una riga di intestazione nel messaggio di risposta HTTP;
2. una riga di intestazione nel messaggio di richiesta HTTP;
3. un file mantenuto sil sistema dell'utente e gestito dal browser;
4. un database sul sito.

Quando un utente visita per la prima volta un sito, il sito crea un identificativo unico e una voce nel proprio database, indicizzata dal numero identificativo. A questo punto il server risponde, includendo nella risposta HTTP l'intestazione *Set-cookie:* che contiene il numero identificativo. Quando il browser del client riceve il messaggio di risposta HTTP, vede l'intestazione *Set-cookie* e aggiunge una riga al file dei cookie che gestisce. Questa riga include il nome dell'host del server e il numero identificativo. Quando l'utente visita nuovamente lo stesso sito, il suo browser consulta il suo file di cookie, estrae il suo numero identificativo per il sito e pone nella richiesta HTTP una riga di intestazione del cookie che include tale numero. Aggiunge la riga di intestazione *Cookie:* seguita dal numero identificativo. I cookie, quindi, possono essere usati per identificare gli utenti. La prima volta che visita un sito, un utente può fornire un'identificazione. Successivamente il browser passa un'intestazione di cookie al server durante tutte le successive visite al sito,

identificando l'utente sul server.
Le informazioni dei cookie sono memorizzate lato client dal browser in uno spazio dedicato (il «cookie store»), che oggigiorno è tipicamente implementato come un database SQLite. Non dovrebbero mai essere utilizzati per memorizzare password o altre informazioni sensibili in chiaro. Quando il server risponde con un'intestazione `Set-Cookie`, può specificare una serie di attributi predefiniti per regolarne il ciclo di vita e la sicurezza, come `Expires`, `Max-Age` (scadenza), `Domain`, `Path`, `Secure` e `HttpOnly`.

3.1.8. Web caching

Una **web cache**, nota anche come **proxy server**, è un'entità di rete che soddisfa richieste HTTP al posto del web server effettivo. Il proxy ha una propria memoria sul disco (una cache) in cui conserva copie di oggetti recentemente richiesti. Il browser di un utente può essere configurato in modo che tutte le richieste HTTP dell'utente vengano innanzitutto dirette al proxy server. Una volta configurato il browser, ogni richiesta di oggetto da parte del browser viene diretta al proxy. Quando un browser richiede un oggetto, accade che:

1. Il browser stabilisce una connessione TCP con il proxy server e invia una richiesta HTTP per l'oggetto specificato.
2. Il proxy controlla la presenza di una copia dell'oggetto memorizzata localmente. Se l'oggetto viene rilevato, il proxy lo inoltra all'interno di un messaggio di risposta HTTP al browser.
3. Se, invece, la cache non dispone dell'oggetto, il proxy apre una connessione TCP verso il server di origine. Quindi, il proxy invia al server una richiesta HTTP per l'oggetto. Una volta ricevuta tale richiesta, il server di origine invia al proxy l'oggetto all'interno di una risposta HTTP.
4. Quando il proxy riceve l'oggetto, ne salva una copia nella propria memoria locale e ne inoltra un'altra copia, all'interno di messaggio di risposta HTTP, al browser.

Il proxy è contemporaneamente server e client: quando riceve richiesta da un browser e gli invia risposte agisce da server, quando invia richieste e riceve risposte da un server di origine funziona da client.
Il web caching si è sviluppato in Internet per due ragioni. Innanzitutto, un proxy può ridurre in modo sostanziale i tempo di risposta alle richieste dei client, in particolare se l'ampiezza di banda che costituisce il collo di bottiglia tra il client e il server di origine è molto inferiore rispetto all'ampiezza di banda minima tra client e proxy. In secondo luogo, i proxy possono ridurre il traffico sul collegamento di accesso a Internet, con il vantaggio di non dover aumentare l'ampiezza di banda frequentemente e ottenere quindi una riduzione dei costi.
Un'altra alternativa al proxy server è il **reverse proxy server** che è un proxy che sta davanti ai server di origine. Un client manda la richiesta al proxy credendo che sia il web server e il proxy la inoltra a uno dei server di orogine, che restano nascosti

al client. Il reverse proxy server è il simmetrico del forward proxy: quello agisce per conto dei client, questo per conto dei server. Il reverse proxy server è importante per la sicurezza perchè nasconde l'indirizzo IP dei server, è importante per il load balancing e per le funzionalità di caching.

3.1.9. HTTP/2

La maggior parte dei browser, supporta HTTP/2. Gli obiettivi principali di HTTP/2 sono quello di ridurre la latenza percepita attivando il multiplexing di richiesta e risposta su singola connessione TCP, di fornire supporto per la priorità delle richieste e il server push e di fornire una compressione efficiente dei campi di intestazione HTTP. HTTP/2 non modifica metodi, codici di stato, URL e campi di intestazione di HTTP, ma cambia il modo in cui i dati vengono formattati e trasportati tra il client e il server. Abbiamo detto che HTTP/1.1 utilizza connessioni TCP persistenti ma l'invio di tutti gli oggetti web su una singola connessione web ha il problema dell'**HEAD OF LINE (HOL) blocking**. Per comprendere il blocco HOL, si consideri una pagina web che include una pagina di base HTML, un grande video clip e molti piccoli oggetti sotto il video. Supponiamo inoltre che ci sia un collegamento a collo di bottiglia con velocità medio-bassa sul percorso tra server e client. Utilizzando una singola connessione TCP, il video clip impiegherebbe molto tempo per passare attraverso il collegamento a collo di bottiglia e gli oggetti piccolo subirebbero un ritardo.
Per aggirare questo problema è possibile aprire più connessioni TCP parallele per trasportare una singola pagina web, «imbrogliando» e prendersi una parte maggiore della larghezza di banda del collegamento. Uno degli obiettivi principali di HTTP/2 è eliminare o ridurre il numero di connessioni TCP parallele per il trasporto di una singola pagina web.
Per implementare il multiplexing e aggirare il blocco HOL, HTTP/2 utilizza il «framing», suddividendo la risposta in frame binari di dimensione limitata (la dimensione massima è negoziata, con un default di 16 KB). Questo permette un «interleaving» (intervallamento) fluido dei frame di oggetti diversi. HTTP/2 (basato in origine sul protocollo SPDY di Google) fornisce anche supporto per la priorità: il client può indicare per ogni richiesta un livello di urgenza da 0 a 7 (RFC 9218). Inizialmente era previsto anche il **server push** (l'invio non richiesto di asset per la pagina), ma questa funzionalità è stata di fatto abbandonata e rimossa dai browser basati su Chromium a partire dal 2022.

3.1.10. HTTP/3

QUIC è un nuovo protocollo di trasporto implementato nel livello di applicazione sul portocollo UDP. QUIC offre diverse opportune funzionalità per HTTP, come il multiplexing dei messaggi (*interleaving*), il controllo di flusso per ogni stream e la creazione di connessioni a bassa latenza. HTTP/3 è un altro, nuovo protocollo

HTTP progettato per funzionare su QUIC. HTTP/3 non è ancora stato completmente standardizzato. Molte delle funzionalità HTTP/2 sono incluse in QUIC, facilitando la progettazione di HTTP/3.
HTTP/3 implementa in modo nativo la sicurezza: integra infatti il protocollo TLS (Transport Layer Security), ovvero lo stesso protocollo alla base delle connessioni HTTPS, garantendo comunicazioni sicure di default.

3.1.11. Streaming video e reti per la distribuzione di contenuti

Lo streaming video rappresenta circa l'80% del traffico Internet (statistica del 2020).Analizziamo come i servizi streaming vengano implementati nell'Internet di oggi.

Video su Internet

Nelle applicazioni di streaming di video registrati, i contenuti sono video, memorizzati su server a disposizione degli utenti su richiesta. Innanzitutto, analizziamo le caratteristiche dei video. Un video è una sequenza di immagini, visualizzate a velocità costante di, ad esempio, 24 o 30 immagini al secondo. Un'immagine non compressa e codificata digitalmente consiste di un array di pixel, ognuno dei quali codificato con un numero di bit per rappresentare luminanza e crominanza. I video possono essere compressi in modo da raggiungere un compromesso tra qualità del video e bit rate. Gli algoritmi di compressione oggi disponibili sono in grado di comprimere un video a qualsiasi bit rate si desideri. Più è alto il bit rate, migliore è la qualità dell'immagine e l'esperienza visiva globale dell'utente.
Dal punto di vista del networking, la caratteristica più importante è l'elevata velocità con cui è necessario inviare i bit sulla rete (**bit rate**). La misura in assoluto più importante per lo streaming video è il throughput medio, il cui valore deve essere almeno pari a quello del bit rate del video per avere una riproduzione continua.
La compressione può essere usata anche per creare versioni multiple dello stesso video, a livello di qualità diversi. Ad esempio, si può usare la compressione per creare tre versioni dello stesso video, a 300 kbps, 1 Mbps e 3 Mbps e gli utenti possono decidere quale versione guardare in funzione della larghezza di banda disponibile.

Streaming HTTP e DASH

Nello streaming HTTP il video viene semplicemente memorizzato in un server HTTP come un file ordinario con un URL specifico. Quando un utente vuole vedere un video, il client stabilisce una connessione TCP con il server e invia una richiesta GET HTTP per il suo URL. Il server invia il file video, all'interno di un messaggio di risposta HTTP, più velocemente possibile dati il traffico e i protocolli di rete. Sul lato client i byte vengono memorizzati in un buffer dell'applicazione client. Quando il numero di byte nel buffer supera una soglia fissata, l'applicazione client inizia la riproduzione: periodicamente prende i frame video dal buffer, li decomprime e li visualizza all'utente. Quindi l'applicazione di video streaming consiste nel visua-

lizzare i frame mentre li riceve, memorizzando nel buffer gli ultimi.
Lo streaming HTTP presenta un ande svantaggio: i client ricevono tutti la stessa versione codificata del video, nonstante abbiano a disposizione una larghezza di banda che può variare sensibilmente da un caso all'altro e nel tempo. Per superare questo problema è stato sviluppato un nuovo tipo di streaming basato su HTTP, chiamato **streaming dinamico adattativo su HTTP**, *Dynamic Adaptive Streming Over HTTP*, detto **DASH**. In DASH, i video vengono codificati in diverse versioni, ognuna avente un bit rate differente e quindi un differente livello di qualità. Il client richiede pezzi di segmenti video lunghi alcuni secondi da versioni differenti in modo dinamico. Quando la banda disponibile è elevata, il client seleziona automaticamente i blocchi da una versione con alto bit rate, mentre quando la banda disponibile è pcoa, seleziona una versione a basso bit rate. Il client seleziona un blocco alla volta con un messaggio di richiesta GET HTTP.
Da una parte, DASH permette a client con accessi a Internet diversi di fare streaming del video con codifiche diverse: client con connessioni lente possono ricevere versioin a basso bit rate e quindi a bassa qualtià, mentre client con connessioni in fibra possono ricevere una versione ad alta qualità. D'altra canto, DASH permette anche a un client di adattare la scelta della versione alla banda disponibile se varia durante la sessione. Tale caratteristica è importante per gli utenti mobili, per i quali la banda disponibile fluttua quando cambiano stazione base.
Con DASH, i video nelle varie versioin sono memorizzati nel server HTTP, ognuno a un URL diverso. Il server HTTP ha anche un file di descrizione, detto **manifest file**, che per ogni versione fornisce il rispettivo URL insieme al bit rate. Il client richiede questo file per venire a conoscenza delle varie versioni disponibili. Quindi seleziona a ogni istante un blocco, specificando nel messaggio di richiesta GET di HTTP un URL e un intervallo di byte. Mentre scarica i blocchi, il client misura la banda di ricezione ed esegue un algoritmo per selezionare il blocco successivo. Naturalmente il client, se ha molti dati del video memorizzati nel buffer e la banda di ricezione misurata è large, sceglie una versione ad alto bit rate per il blocco successivo; al contrario, se ha pochi dati nel buffer e banda bassa, sceglie una versione basso rate. Pertanto DASH permette al client di cambiare liberamente il livello di qualità.

Reti per la distribuzione di contenuti

Oggigiorno molte aziende di video in Internet distribuiscono quotidianamente video on demand con streaming dell'ordine dei Mbps a milioni di utenti. Mandare in streaming una tale mole di traffico a utenti sparsi in tutot il mondo, fornendo riproduzione continua e alta interattività è complesso. L'approccio più diretto sarebbe costruire un unico enorme data center, memorizzare in esso tutti i video e mandarli in streaming direttamente. Questo approccio però presenta tre grandi problemi:

1. Se il client è lontano dal data center, i pacchetti dal server al client devono percorrere un lungo cammino passando per molti ISP, magari in continenti diversi.

Se uno dei collegamenti ha un throughput minore della velocità di consumo del video, anche il throughput totale end-to-end lo sarà, causando continui e fastidiosi fermi immagine.
2. Un video molto popolare verrebbe inviato molte volte sullo stesso collegamento, non solo sprecando banda, ma constrigendo anche l'azienda a pagare tutte le volte il suo ISP per inviare gli stessi dati.
3. Un singolo data center rappresenta un singolo punto di rottura: se il data center o il collegamento da esso a Internet si interrompesse, l'azienda non sarebbe più in grado di distribuire alcun video.

Per superare questi problemi, quasi tutte le maggiori aziende di video streaming usano le **CDN** (*Content Distribution Networks*). Una CDN gestisce server distribuiti in molti posti diversi, memorizza copie dei video e di altri contenuti web nei server e cerca di dirigere le richieste degli utenti al punto della CDN in grado di offrire il servizio migliore. La CDN può usare una **CDN privata**, cioè proprietaria del fornitore di contenuti. Alternativamente può essere usata la **CDN di terze parti** che distribuisce contenuti per conto di molti fornitori di contenuti. Le CDN generalmente adottano una delle due politiche di dislocazione dei server che seguono:

- **Enter deep** («entrare in profondità»). Una filosofia è quella di entrare in profondità nelle reti di accesso degli ISP installando gruppi di server, detti anche cluster, negli ISP di accesso sparsi in tutto il mondo. Questo approccio viene usato distribuendo server in migliaia di posti diversi. L'obiettivo è quello di essere vicini agli utenti finali in modo da migliorare il ritardo percepito dall'utente e il throughput, diminuendo il numero di collegamenti e router tra l'utente finale e il cluster CDN da cui riceve i contenuti. Questo approccio pone però grandi sfide nella manuntenzione e gestione dei cluster.
- **Bring home** («portare a casa»). Il secondo approccio è quello di portarsi in caso l'ISP costruendo grandi cluster in pochi punti chiave e interoconneterli usando una rete privata ad alta velocità. invece di entrare negli ISP di accesso, queste CDN pongono i loro cluster negli IXP. Tale approccio presenta meno problemi di gestione e manutenzione, ma spesso anche una minore qualità di servizio.

Una volta posizionati i cluster, la CDN replica i contenuti su di essi. I video raramente richiesti non vengono copiati su tutti i cluster.
Molte CDN non spingono i loro video verso i cluster, ma usano una semplice strategia per cui se un client richiede un video a un cluster che non lo ha memorizzato, il cluster recupera il video da un archivio centrale o da un altro cluster e ne memorrizza localmente una copia mentre lo manda in streaming al client.

Come funziona la CDN

Quando un browser nell'host di un utente chiede il recupero di uno specifico video identificato da un URL, la CDN deve interecettare la richiesta in modo d poter

1. determinare il cluster di server più appropriato per quel client a quell'istante
2. dirigere la richiesta del client a uno dei server di quel cluster.

Esaminiamo i meccanismi che stanno dietro l'intercettazione e il reindirizzamento di una richiesta, discutendo poi come una CDN decida quale sia il cluster più adatto. Molte CDN sfruttano il servizio DNS per intercettare e ridirigere le richieste. Consideriamo un esempio dove un fornitore di contenuti, *NetCinema*, impieghi una CDN di terza parte, *KingCDN*, per distribuire i suoi video. A ogni video viene assegnato un URL che include la stringa «video» e un identificatore univoco per tale video. Vengono compiuti 6 passi:

1. L'utente visita la pagina web di *NetCinema*;
2. Quando l'utente seleziona il link *http://video.netcinema.com/6Y7B23V* , il suo host invia un'interrogazione DNS a *video.netcinema.com*
3. Il **DNS server locale (LDNS)** dell'utente invia l'interrogazione DNS a un DNS server autoritativo per *NetCinema* che osserva la stringa «video» nel nome dell'host *video.netcinema.com*. Il DNS server autoritativo per *NetCinema* per passare l'interrogazione DNS a *KingCDN*, invece di restituire un indirizzo IP, restituisce all'LDNS il nome di un host nel dominio di *KingCDN*.
4. Da questo momento l'interrogazione DNS entra nell'infrastruttura DNS privata di *KingCDN*. Lo LDNS dell'utente invia una seconda interrogazione e infine il sistema DNS di *KingCDN* restituisce gli indirizzi IP di un server *KingCDN* all'LDNS. È quindi qui, all'interno del sistema DNS di *KingCDN*, che viene specificato il server CDN da cui il client riceverà il contenuto.
5. LDNS inoltra l'indirizzo IP del nodo CDN che fornirà il contenuto all'host dell'utente.
6. Il client, una volta ricevuto l'indirizzo IP del server di *KingCDN*, stabilisce una connessione TCP diretta con il server a quell'indirizzo IP e gli invia una richiesta GET HTTP per il video. Nel caso venga impiegato DASH, il server innanzitutto inverià al client il file manifesto con una lista di URL.

Strategie di selezione dei cluster

Il cuore dell'installazione di una CDN è la **strategia di selezione dei cluster**: un meccanismo per dirigere dinamicamente i client a un cluster di server o a un data center della CDN. La CDN apprende l'indirizzo IP del server LDNS del client attraverso la richiesta DNS del client. Dopo aver appreso tale indirizzo IP, la CDN deve selezionare un cluster appropriato. Le strategie di selezione del cluster sono generalmente proprietarie.

Una semplice strategia consiste nell'assegnare a un client il cluster *geograficamente più vicino*. Usando un database commerciale di geo-localizzazione, gli indirzzi IP degli LDNS vengono associati a un luogo geografico. La CDN, quando riceve una richiesta DNS da un particolare LDNS, sceglie il cluster a esso più vicino geograficamente in linea d'aria. Tale soluzione potrebbe non andare bene per alcuni client,

in quanto il cluster più vicino potrebbe essere diverso da quello più vicino dal punto di vista del percorso in rete. Un altro problema derivato da tutti gli approcci basati sul DNS deriva dal fatto che alcunu utenti sono configurati per usare LDNS remoti. Inoltre questa strategia non tiene conto delle variazioni temporali del ritardo di rete e della banda disponibile.
Un secondo approccio determina il cluster migliore per un client basandosi sulle condizioni di traffico correnti, effettuando **misure in tempo reale** delle prestazioni di ritardo e perdita tra cluster e client. Un problema di questo approccio è che molti LDNS sono configurati per non rispondere a tali richieste.

3.2. DNS: il servizio directory di Internet

Gli host Internet possono essere identificati in vari modi. Gli **hostname** quali *www.facebook.com*, *www.google.com*, risultano abbastanza approprati per le persone, ma forniscono poca informazione sulla loro collocazione all'interno di Internet.
Un nome quale *www.eurecom.fr*, che termina con il suffisso del paese, *.fr*, ci dice che l'host si trova probabilmente in Francia, ma niente di più. Inoltre, dato che questi nomi sono costituiti da un numero variabile di caratteri alfanumerici, sarebbero difficilmente elaborabili dai server. Per tali motivi, gli host vengono identificati dai cosiddetti **indirizzi IP**.
Un indirizzo IP consiste di quattro byte e presenta una rigida struttura gerarchica. Ha una forma del tipo 121.7.106.83, in cui ogni punto separa uno dei byte espressi con un numero decimale compreso tra 0 e 255. Un indirizzo IP è gerarchico perchè, leggendolo da sinistra a destra, otteniamo informazioni semper più specifiche sulla collocazione dell'host in Internet.

3.2.1. Servizi forniti da DNS

Quindi, esistono due modi per identificare gli host: il nome e l'indirizzo IP. Le persone preferiscono il primo, mentre i router prediligono gli indirizzi IP a lunghezza fissa e strutturati in modo gerarchico. Al fine di conciliare i due approcci è necessario un servizio che traduce gli hostname nei loro indirizzi IP (o viceversa). Si tratta del principale compito del *domain name system*, **DNS**. DNS è

1. un database distribuito implementato in una gerarchia di **DNS server**
2. un protocollo a livello di applicazione che consente agli host di interrogare il database.

I DNS server sono generalmente macchine UNIX che eseguono un software chiamato BIND, Berkley Internet Name Domain. Il protocollo DNS utilizza UDP e la porta 53 per mantenere una bassa latenza. Il protocollo DNS utilizza, invece, il protocollo TCP sulla porta 53 quando ci sono molti dati, che superano i 512 byte, da trasferire in modo affidabile. L'estensione EDNS permette oggi di inviare risposte UDP più grandi, fino a 1232 byte, limitando la necessità di passare a TCP.

DNS viene utilizzato da altri protocolli a livello di applicazione, tra cui HTTP e SMTP, per tradurre i nomid i host forniti dall'utente in indirizzi IP.
Analizziamo cosa succede quando un browser in esecuzione sull'host di un utente richiede l'URL *www.someschool.edu/index.html*. L'host dell'utente, per essere in grado di inviare un messaggio di richiesta HTTP al web server *www.someschool.edu*, deve ottenere prima il suo indirizzo IP. Ciò avviene come segue.

1. La stessa macchina utente esegue il lato client dell'applicazione DNS.
2. Il browser estrae il nome dell'host, *www.someschool.edu*, dall'URL e lo passa al lato client dell'applicazione DNS.
3. Il client DNS invia un'interrogazione contenente l'hostname a un DNS server.
4. Il client DNS prima o poi riceve una risposta, che include l'indirizzo IP corrispondente all'hostname.
5. Una volta ricevuto l'indirizzo IP dal DNS, il browser può dare inizio a una connessione TCP verso il processo server HTTP collegato alla porta 80 di quell'indirizzo IP.

Il DNS introduce un ritardo aggiuntivo, talvolta sostanziale, alle applicazioni Internet che lo utilizzano. Per evitare ciò, l'indirizzo IP desiderato si trova spesso nella cache di un DNS server vicino, il che aiuta a ridurre il traffico DNS in rete e il ritardo medio del servizio.
Oltre alla traduzione degli hostname in indirizzi IP, DNS mette a disposizione altri importanti servizi.

1. **Host aliasing**. Un host dal nome complicato può avere uno o più sinonimi, detti *alias*. Per esempio, *relay1.west-coast.enterprise.com* potrebbe avere, diciamo, due sinonimi quali *enterprise.com* e *www.enterprise.com*. In questo caso, si dice che il nome *relay1.west-coast.enterprise.com* è un **hostname canonico**. Gli alias sono generalmente più facili da ricordare rispetto agli hostname canonici. Il DNS può essere invocato da un'applicazione per ottenere l'hostname canonico di un sinonimo.
2. **Mail server aliasing**. È auspicabile che gli indirizzi di posta elettronica siano facili da ricordare. Ad esempio se un'utente ha un account Hotmail, il suo indirizzo di posta elettronica potrebbe essere «bob@yahoo.com». Tuttavia, l'hostname del server di posta Hotmail è molto più complicato da ricordare rispetto a *yahoo.com*. Ad esempio, il nome canonico potrebbe essere simile a *relay1.west-coast.yahoo.com*. Un'applicazione di posta può invocare il DNS per ottenere il nome canonico di un sinonimo fornito.
3. **Distribuzione del carico di rete (load distribution)**. Il DNS viene utilizzato per distribuire il carico tra server replicati, per esempio dei web server. I siti con molto traffico, vengono replicati su più server, ognuno eseguito su un host diverso con indirizzo IP differente. Nel caso di web server replicati, va associato a ogni hostname canonico un insieme di indirizzi IP. Il database DNS contiene

questo insieme di indirizzi. Quando i client effettuano una query DNS per un nome associato a un insieme di indirizzi, il server risponde con l'intero insieme di indirizzi, ma ne varia l'ordinamento a ogni risposta. Dato che un client invia il suo messaggio di richiesta HTTP al primo indirizzo IP elencato nell'insieme, la rotazione DNS distribuisce il traffico sui server replicati. La rotazione viene anche utilizzate affinchè più server di posta possano condividere lo stesso nome. Inoltre, le società di distribuzione dei contenuti hanno fatto uso del DNS in mdoi più sofisticati per effettuare la diffusione di contenuti web.

DNS è specificato nelle RFC 1034 e 1035 ed è stato aggiornato nelle successive RFC aggiuntive.

3.2.2. Panoramica del funzionamento di DNS

Analizziamo il servizio di traduzione da hostname a indirizzo IP.
Supponiamo che una certa applicazione in esecuzione sull'host di un utente abbia necessità di trudurre un hostname in indirizzo IP. L'applicazione invocherà il lato client del DNS, specificando l'hostname da tradurre. Su molte macchine basate su UNIX, `gethostbyname()` è la chiamata di funzione effettuata da un'applicazione per ottenere il servizio di traduzione. Il DNS sull'host prende poi il controllo, inviando un messaggio di richiesta sulla rete. Tutte le query DNS e i messaggi di risposta vengono inviati all'interno di datagrammi UDP diretti alla porta 53. Dopo un ritardo che varia dai millisecondi ai secondi, il client DNS sull'host dell'utente riceve un messaggio di risposta contenente la corrispondenza desiderata, che viene poi passata all'applicazione che ne ha fatto richiesta. Dal punto di vista dell'applicazione, il DNS è una scatola nera che fornisce un servizio di traduzione semplice e diretto. La scatola nera è costituita da un gran numero di DNS server distribuiti per il mondo e da un protocollo a livello di applicazione che specifica la comunicazione tra DNS server e host richiedenti.
Un primo approccio potrebbe essere quello di utilizzare un DNS server contenente tutte le corrispondenze. Questa semplicità progettuale è inappropriata per l'attuale Internet, dotata di un vasto e sempre crescente numero di host.
Tra i problemi legati a uno schema centralizzato ricordiamo i seguenti.

- **Un solo point of failure** (*punto di vulnerabilità*). Se il DNS server si guasta, ne soffre l'intera Internet.
- **Volume di traffico**. Un singolo DNS server dovrebbe gestire tutte le richieste.
- **Database centralizzato distante**. Un singolo DNS server non può essere vicino a tutti i client.
- **Manutenzione**. Il singolo DNS server dovrebbe contenere record relativi a tutti gli host di Internet. Non solo tale database centralizzato sarebbe vasto, ma dovrebbe essere aggiornato frequentemente per tener conto di ogni nuovo host.

Un database centralizzato su un singolo DNS server non è in grado di adattarsi alla crescita esponenziale della rete. Quindi, il DNS è stato progettato in maniera distribuita.

Un database distrubuito e gerarchico

Per trattare il problema della scalabilità, il DNS utilizza un grande numero di server, organizzati in maniera gerarchica e distribuiti nel mondo. In prima approssimazione, esistono tre classi di DNS server: i **root server**, i **top-level domain (TLD) serve** e i **server autoritativi**, organizzati in una gerarchia, come mostrato nella seguente figura.

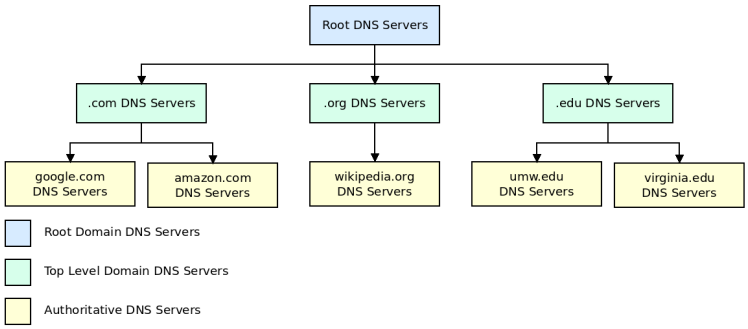


Figura 5: Gerarchia parziale dei server DNS

Per comprendere l'interazione tra le classi, supponiamo che un client DNS voglia determinare l'indirizzo IP relativo all'hostname `www.amazon.com`. Per fare ciò, il client contatta uno dei root server, che gli restituisce uno o più indirizzi IP relativi al TDL server per il dominio `com`. Contatta uno di questi TLD server,che gli restituisce uno o più indirizzi IP del server autoritativo per `amazon.com`. Infine, contatta uno dei server autoritativi per `amazon.com`, che gli restituisce l'indirizzo IP dell'hostname `www.amazon.com`. Analizziamo le tre classi di DNS server.

- **Root server**. In Internet esistono più di 1000 root server, dislocati in tutto il mondo. Tali root server sono copie di 13 differenti root server gestiti da 12 diverse organizzazioni, coordinate attraverso la *Internet Assigned Numbers Authority*. I root server forniscono gli indirizzi IP dei TLD server.
- **Top-level domain (TLD) server**. Questi server si occupano dei domini di primo livello quali com, org, net, edu e gov, e di tutti i domini di primo livello relativi ai vari paesi, come it, uk, fr, ca e jp. I TLD server forniscono gli indirizzi IP dei server autoritativi. L'assegnazione dei domini che indicano una nazione o un tipo di organizzazione è gestita dall'ICANN (Internet Corporation for Assigned Names and Numbers).
- **DNS server autoritativi**. Ogni organizzazione dotata di host pubblicamente accessibili tramite Internet deve fornire record DNS pubblicamente accessibili che associno i nomi di tali host a indirizzi IP. Il DNS server autoritativo dell'organizzazione ospita questi record. Un'organizzazione può scegliere di im-

plementare il proprio server autoritativo o di pagare un fornitore di serverizi per ospitare questi record su un suo server. La maggior parte delle università e delle grandi società implementa e gestisce dei propri server autoritativi primario e secondario.

I DNS server appena descritti sono tutti collocati in una gerarchia di DNS server. Esiste un altro importante tipo di DNS, detto **DNS server locale**, che non appartiene strettamente alla gerarchia di server, ma che è comunque centrale nell'architettura DNS. Ciascun ISP ha un DNS server locale, detto anche **default name server**. Quando un host si connette a un ISP, quest'ultima gli fornisce un indirizzo IP tratto da uno o piu dei suoi DNS server locali, generalmente tramite DHCP. Si può facilmente determinare l'indirizzo IP del proprio DNS server locale accedendo alla finestra di stato della rete in Windows o UNIX. Un DNS server locale è solitamente «vicino» all'host. Quando un host effettua una richiesta DNS, la query viene inviata al DNS server locale, che opera da proxy e inoltra la query alla gerarchia dei DNS server.
Le richieste DNS possono essere di due tipi:

- **query ricorsive**: è una richiesta in cui un client chiede a un server DNS (detto *resolver ricorsivo*) di ottenere l'indirizzo IP completo e definitivo per un nome di dominio, oppure di restituire un errore se non lo trova. Il server ha tutta la responsabilità: se non ha il dato in cache, contatta altri server per conto del client finchè non ottiene la risposta definitiva e gliela consegna.
- **query iterativa**: il server DNS interpellato non fa tutto il lavoro per conto del richiedente. Se conosce la risposta la fornisce, altrimenti, resituisce la «migliore risposta possibile», ovvero un **rinvio (referral)**, indicando al client quale altro server DNS conviene contattare subito dopo. Il client stesso prosegue l'esplorazione passo dopo passo.

I root server e i TLD server sono generalmente configurati per essere non-ricorsivi e non accettano query ricorsive a causa dell'eccessivo traffico.

DNS Caching

Il **DNS caching** è una caratteristica di fondamentale importanza. Il DNS sfrutta in modo estensivo il caching per migliorare le prestazioni di ritardo e per ridurre il numero di messaggi DNS che «rimbalzano» su Internet. L'idea del DNS caching è semplice. In una concatenazione di richieste, il DNS server che riceve una risposta DNS può mettere in cache le informazioni contenute.
Se una ⟨hostname/indirizzo IP⟩ è nella cache di un DNS server e giunge al server un'altra richiesta con lo stesso hostname, il DNS server può fornire l'indirizzo IP desiderato, anche se non è autoritativo per tale indirizzo. Dato che gli host e le associazioni tra nome e indirizzo IP non sono in alcun modo permanenti, i DNS server invalidano le informazioni in cache dopo un periodo di tempo fissato. Un DNS

server locale, può, inoltre, memorizzare in cache gli indirizzi IP dei TLD server, consentendogli di aggirare i root server nella catena di richieste.

3.2.3. Record e messaggi DNS

I server che implementano il database distribuito di DNS memorizzano i cosiddetti **record di risorsa** (RR, *resource record*), tra cui quelli che forniscono le corrispondenze tra nomi e indirizzi. Ogni messaggio di risposta DNS trasporta uno o più record di risorse. Un record di risorsa contiene i seguenti campi:

(Name, Value, Type, TTL)

TTL è il time to live, ossia il tempo residuo di un record e determina quando una risorsa vada rimossa dalla cache. Il significato di `Name` e `Value` dipene da `Type`:

- Se `Type=A`, allora `Name` è il nome dell'host e `Value` è il suo indirizzo IP. Pertanto un record di tipo A fornisce la corrispondenza tra hostname standard e indirizzo IPv4.
- Se `Type=AAAA`, allora `Name` è il nome dell'host e `Value` è il suo indirizzo IPv6.
- Se `Type=NS`, allora `Name` è un dominio e `Value` è l'hostname del DNS server autoritativo che sa come ottenere gli indirizzi IP degli host nel dominio. Questo record viene usato per instradare le richieste DNS successive alla prima nella concatenazione delle query.
- Se `Type=CNAME`, allora `Value` rappresenta il nome canonico dell'host per il sinonimo `Name`. Questo record può fornire agli host richiedenti il nome canonico relativo a un hostname.
- Se `Type=MX`, allora `Value` è il nome canonico di un mail server che ha il sinonimo `Name`. Questo tipo di record consente agli hostname dei mail server di avere sinonimi semplici. Per ottenere il nome canonico del server di posta, un client DNS dovrebbe interrogare un record MX. Per ottenere il nome canonico dell'altro server, il client DNS dovrebbe interrogare il record CNAME.

Un DNS server autoritativo per un certo hostname contiene un record di tipo A per l'hostname. Anche se il server non fosse autoritativo, potrebbe tuttavia contenere un record di tipo A nella propria cache. Se un server non è quello autoritativo per un certo hostname, allora conterrà un record di tipo NS per il dominio che include l'hostname, e conterrà anche un record di tipo A che fornisce l'indirizzo IP del DNS server nel campo `Value` del record NS.

Messaggi DNS

Abbiamo fatto riferimento agli unici due tipi di messaggio DNS: le query e i messaggi di risposta che presentano, entrambi, lo stesso formato, mostrati nella seguente figura.

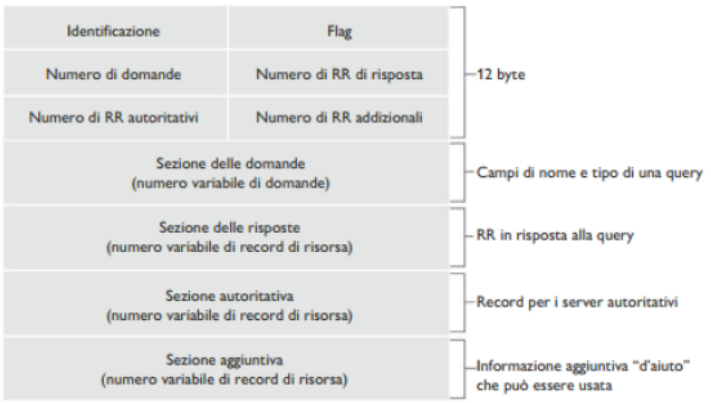


Figura 6: Formato dei messaggi DNS

La semantica dei campi dei messaggi DNS è la seguente.

- I primi 12 byte rappresentano la *sezione di intestazione (header section)*, che a sua volta contiene un certo numero di campi.
  - Il primo è un numero di 16 bit che identifica la richiesta. Tale identificatore viene copiato nei messaggio di risposta a una richiesta, consentendo al client di far corrispondere le risposte ricevute con le query inviate.
  - Il campo flag è composto da una serie di bit: il primo, il bit di richiesta/risposta indica se è il messaggio è una richiesta (0) o una risposta (1). Un bit viene impostato nei messaggi di risposta quando il DNS server è autoritativo per il nome richiesto. Un ulteriore bit, chiamato **richiesta di ricorsione**, viene impostato quando un client desidera che il DNS server effettui ricorsione quando non dispone del record. Si imposta un campo da 1 bit di ricorsione disponibile all'interno di una risposta se il DNS server supporta la ricorsione.

  Nell'intestazione troviamo anche quattro campi il cui nome comincia con «numero di», che indicano il numero di occorrenze delle quattro sezioni di tipo dati che seguono l'intestazione.
- La *sezione delle domande* contiene informazioni sulle richieste che stanno per essere effettuate. Inoltre, include
  - un campo nome con il nome che sta per essere richiesto;
  - un campo tipo che indica il tipo della domanda sul nome.
- In una risposta proveniente da DNS server, la *sezione delle risposte* contiene i record di risorsa relativi al nome originariamente richiesto. Una risposta può restituire più RR, dato che un hostname può avere più indirizzi IP. È il caso dei web server replicati.
- La *sezione autoritativa* contiene i record di altri server autoritativi.
- La *sezione aggiuntiva* racchiude altri record utili: ad esempio, se il campo di risposta relativo a una richiesta MX contiene un record di risorsa che fornisce

l'hostname canonico del server di posta, la sezione aggiuntiva contiene un record di tipo A che fornisce l'indirizzo IP relativo all'hostname canonico del server di posta.

3.2.4. Registrazione di un dominio e Zone Transfer

Per inserire un nuovo dominio nel database DNS (ad esempio `networkutopia.com`), è necessario rivolgersi a un **registrar**. Un registrar è un'azienda commerciale, tipicamente accreditata dall'ICANN (Internet Corporation for Assigned Names and Numbers), che verifica l'unicità del nome di dominio, richiede un compenso per il servizio e si occupa di inserirlo nel sistema DNS.
Al momento della registrazione, l'utente deve fornire al registrar i nomi e gli indirizzi IP dei propri DNS server autoritativi, generalmente un primario e un secondario (es. `dns1.networkutopia.com` e `dns2.networkutopia.com`, insieme ai relativi IP). Il registrar provvede a inserire nei TLD server di competenza (ad esempio i server responsabili per il suffisso `.com`) due tipologie di record: i record di tipo NS e quelli di tipo A. Nello specifico, per il server primario dell'esempio verrebbero inseriti i seguenti record di risorsa:

- `(networkutopia.com, dns1.networkutopia.com, NS)`
- `(dns1.networkutopia.com, 212.2.212.1, A)`

Successivamente, è responsabilità del proprietario del dominio assicurarsi che nei propri server autoritativi vengano inseriti i record di risorsa specifici per i servizi ospitati. Ad esempio, deve essere configurato il record di tipo A per il server web (`www.networkutopia.com`) e il record MX per il server di posta elettronica (`mail.networkutopia.com`). L'aggiunta di questi record può avvenire configurando staticamente i server oppure, più recentemente, in modo dinamico inviando messaggi DNS grazie all'opzione UPDATE (specificata nelle RFC 2136 e 3007).
Completata la configurazione, la risoluzione procede normalmente: il DNS server locale di un utente che vuole visitare il sito interrogherà il TLD server, il quale risponderà fornendo l'IP del server autoritativo grazie ai record precedentemente inseriti dal registrar. Interrogando infine il server autoritativo, il client otterrà l'indirizzo IP del web server desiderato.

Zone Transfer

Per garantire disponibilità e affidabilità, i record DNS appena creati devono essere replicati. I record vengono copiati dal server autoritativo primario a quelli secondari tramite un'operazione chiamata **Zone Transfer**. Essendo un'operazione che comporta spesso il trasferimento di un'intera zona e di molti dati (superando il classico limite di 512 byte del protocollo UDP), il trasferimento avviene in modo affidabile su protocollo TCP. Le query utilizzate dai server secondari per allinearsi al primario sono principalmente due:

- **AXFR**: per richiedere una copia completa e integrale dei record della zona.

- **IXFR**: per richiedere un trasferimento incrementale, ottenendo solamente le modifiche più recenti.